Data Warehousing with Azure Synapse (Serverless + Delta Lake)

BootCamp_Project-3

Student Name: Bhaskar Karri

Program: Azure Data Engineering

Table of Contents

1. Project Overview....3
2. Objective....3
3. Solution Architecture....3
Architecture Explanation....4
4. Tools & Technologies....4
5. Data Description....5
6. Implementation Steps....6
Step 1: Data Ingestion (Bronze Layer)....6
Step 2: Data Processing (Silver Layer using Data Flows)....6
Transformations Applied:....6
Output:....7
Step 3: SCD Type 1 Implementation (Gold Layer using Data Flows)....8
Approach Used:....8
md5(columns())....8
Logic:....8
Output:....8
Step 4: Query Delta Data using Serverless SQL Pool....9
Step 5: Data Modeling (Logical Star Schema)....10
Dimension: Customer....10
Dimension: Product....11
Fact Table (Logical View)....11
Step 5: Business-Level Views....12
Customer Sales Analysis....12
Product Performance....13
Sales Trend Analysis....13
Step 6: Security Configuration....14
Step 7: SSMS Connectivity....14
Step 8: Power BI Integration....15
Loaded Views:....15
7. Dashboard Insights....16

8. Challenges Faced..... 16
9. Solutions Implemented..... 16
10. Key Learnings..... 17
11. Final Conclusion..... 17

1. Project Overview

This project focuses on building a modern, scalable data analytics solution using a lakehouse architecture on Azure. The goal was to enable efficient querying and reporting of large-scale transactional data without the need for traditional data warehousing infrastructure.

A serverless approach was adopted using Azure Synapse Serverless SQL Pool, which allows querying data directly from storage without provisioning dedicated resources. The data is stored in Delta Lake format within Azure Data Lake Storage Gen2 (ADLS Gen2), ensuring reliability, performance, and schema consistency.

The processed data is exposed through SQL-based views, enabling seamless integration with Power BI for real-time analytics and visualization.

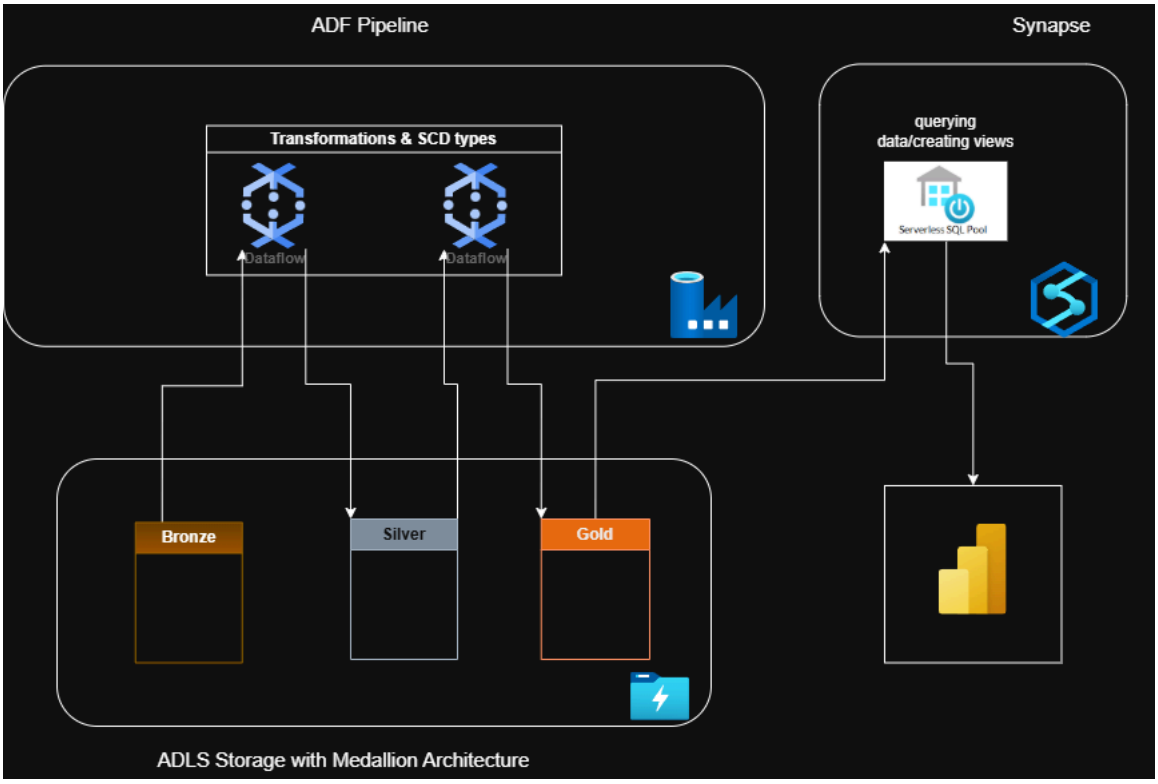
2. Objective

To design a solution that:

- Centralizes data in a single source of truth (data lake)
- Supports on-demand querying without data movement
- Enables real-time and cost-efficient analytics
- Integrates easily with visualization tools like Power BI

3. Solution Architecture

The architecture follows the **Medallion Architecture** (Bronze → Gold) pattern, ensuring clean data processing and optimization.



Architecture Explanation

- **Bronze Layer:** Stores raw, unprocessed data
- **Gold Layer:** Stores cleaned and transformed data in Delta format
- **Serverless SQL Pool:** Queries data directly from ADLS
- **Views Layer:** Implements business logic and star schema
- **Power BI:** Consumes data in real time

4. Tools & Technologies

Tool	Purpose

Azure Synapse Analytics	Query engine (Serverless SQL Pool)
ADLS Gen2	Scalable data storage
Apache Spark	Data transformation and cleaning
Delta Lake	Optimized storage format with ACID properties
Power BI	Data visualization and reporting

5. Data Description

The dataset is from https://www.kaggle.com/datasets/vinothkannaece/sales-dataset represents **sales transactions** and includes the following attributes:

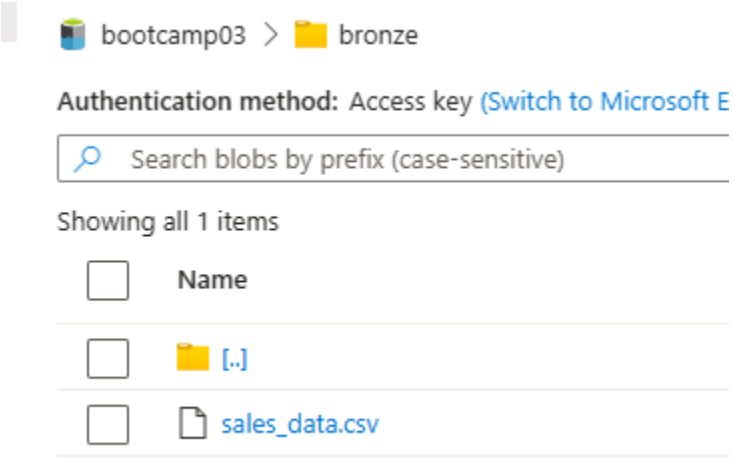
- Product_ID – Unique identifier for products
- Sale_Date – Transaction date
- Region – Sales region
- Sales_Amount – Revenue generated
- Quantity_Sold – Number of units sold
- Product_Category – Product classification
- Unit_Cost – Cost per unit
- Unit_Price – Selling price
- Customer_Type – Customer segmentation
- Discount – Applied discount
- Payment_Method – Mode of payment
- Sales_Channel – Online/Offline channel
- SalesRep – Sales representative

6. Implementation Steps

Step 1: Data Ingestion (Bronze Layer)

- Raw CSV files were ingested into ADLS Gen2
- No transformations were applied at this stage
- Data was stored in its original format for traceability and auditing

Purpose: Preserve raw data for reprocessing if needed

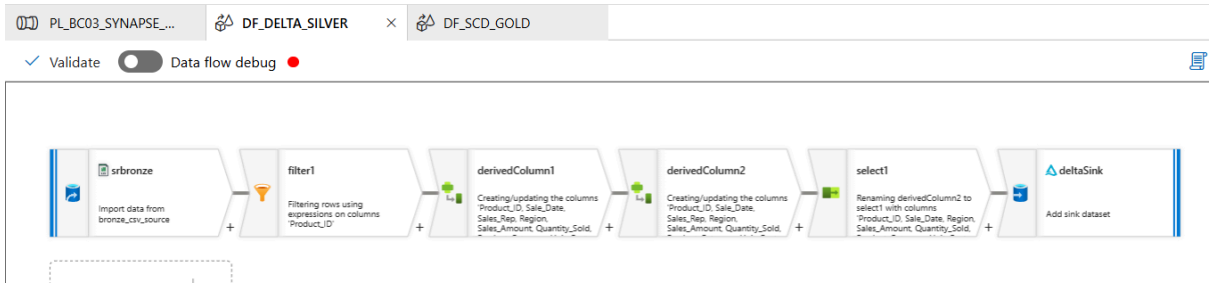


Step 2: Data Processing (Silver Layer using Data Flows)

- Data was extracted from the Bronze layer
- Transformation was performed using Azure Data Factory / Synapse Data Flows

Transformations Applied:

- Removed duplicate records
- Filtered null or invalid values
- Standardized column naming conventions
- Applied basic data type casting
- Cleaned inconsistent or malformed data

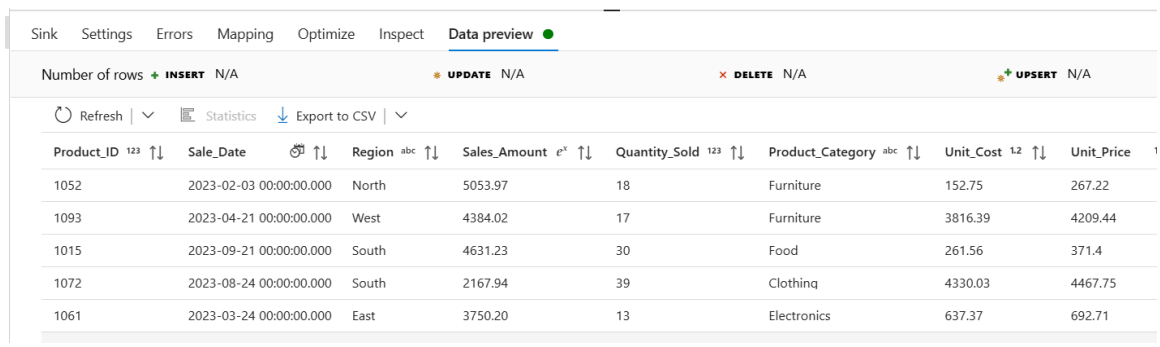


Output:

- Processed data stored in Silver layer in ADLS
- Format: Delta (based on implementation)

Purpose:

To create a clean, standardized, and reliable dataset that serves as the foundation for downstream transformations.

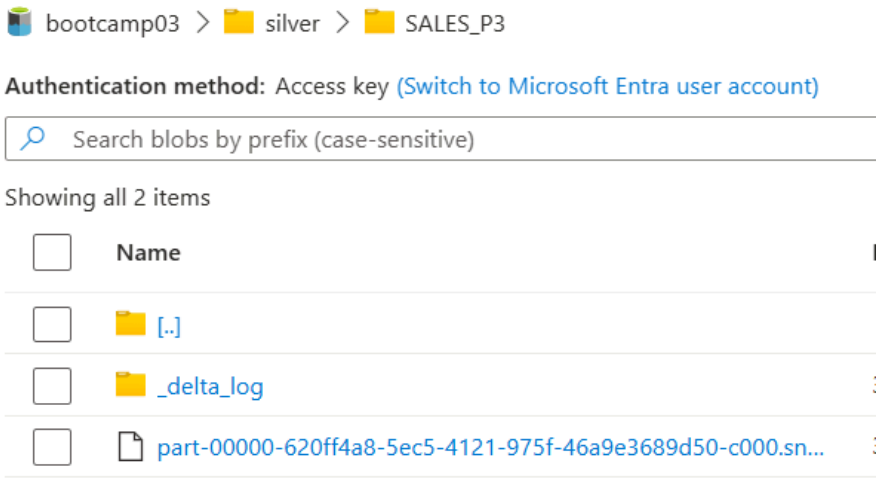


Sink | Settings | Errors | Mapping | Optimize | Inspect | Data preview

Number of rows + INSERT N/A UPDATE N/A DELETE N/A UPSERT N/A

Refresh | Statistics | Export to CSV

Product_ID	Sale_Date	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Price
1052	2023-02-03 00:00:00.000	North	5053.97	18	Furniture	152.75	267.22
1093	2023-04-21 00:00:00.000	West	4384.02	17	Furniture	3816.39	4209.44
1015	2023-09-21 00:00:00.000	South	4631.23	30	Food	261.56	371.4
1072	2023-08-24 00:00:00.000	South	2167.94	39	Clothing	4330.03	4467.75
1061	2023-03-24 00:00:00.000	East	3750.20	13	Electronics	637.37	692.71



Step 3: SCD Type 1 Implementation (Gold Layer using Data Flows)

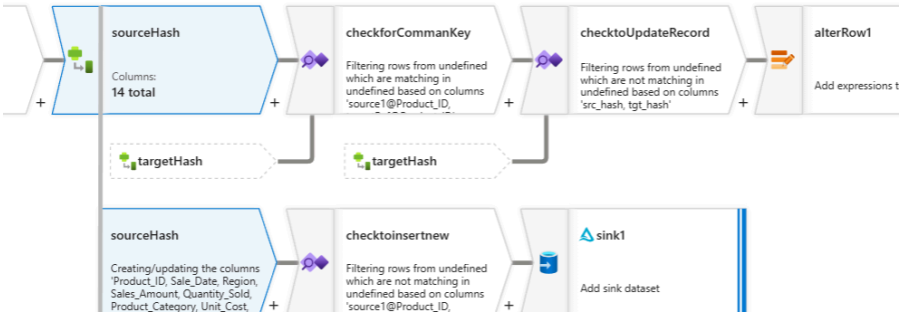
- Data was read from the Silver layer
- Implemented Slowly Changing Dimension (SCD Type 1) using Data Flows

Approach Used:

- Created a hash column to detect changes:

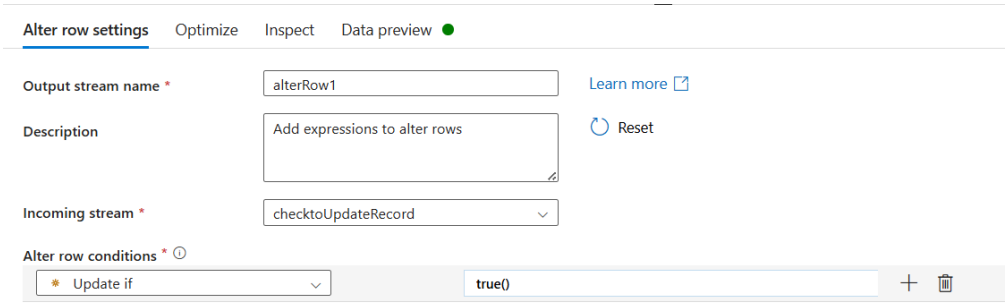
```
md5(columns())
```

- Compared source hash vs target hash
- If matched → No update
- If different → Overwrite existing record (SCD Type 1)



Logic:

- Used Alter Row transformation:
  - Updateif for changed/new records

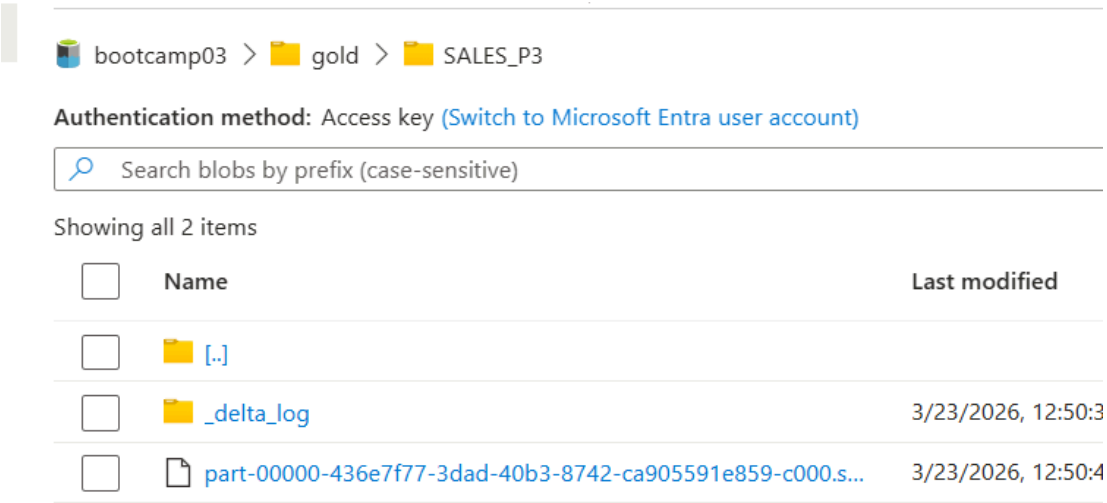


Output:

- Final curated dataset stored in Gold layer
- Format: Delta Lake

Purpose:

- Maintain latest version of records
- Ensure data consistency without duplication
- Prepare data for analytics and reporting



Step 4: Query Delta Data using Serverless SQL Pool

A base view was created to read Delta data directly:

```
CREATE EXTERNAL TABLE ext_sales_delta

(

    Product_ID INT,

    Sale_Date DATETIME2,

    Region VARCHAR(50),

    Sales_Amount FLOAT,

    Quantity_Sold INT,

    Product_Category VARCHAR(50),
```

```
    Unit_Cost FLOAT,

    Unit_Price FLOAT,

    Customer_Type VARCHAR(50),

    Discount FLOAT,

    Payment_Method VARCHAR(50),

    Sales_Channel VARCHAR(50),

    SalesRep VARCHAR(100)

)

WITH (

    LOCATION = '/gold/external/',

    DATA_SOURCE = sql_bc03_views,

    FILE_FORMAT = DeltaFormat

);
```

This acts as the foundation layer for all downstream transformations.

Step 5: Data Modeling (Logical Star Schema)

Since Serverless SQL Pool does not support physical tables efficiently, a logical star schema was implemented using views.

Dimension: Customer

```
CREATE OR ALTER VIEW vw_dim_customer

AS

SELECT DISTINCT

    DENSE_RANK() OVER (ORDER BY Customer_Type, Region) AS
CustomerKey,

    Customer_Type,
```

```
    Region,
    CONCAT(Customer_Type, ' - ', Region) AS CustomerName
FROM vw_sales_base;
```

Dimension: Product

```
CREATE OR ALTER VIEW vw_dim_product
AS
SELECT
    DENSE_RANK() OVER (ORDER BY Product_ID) AS ProductKey,
    Product_ID,
    Product_Category,
    AVG(Unit_Cost) AS AvgCost,
    AVG(Unit_Price) AS AvgPrice
FROM vw_sales_base
GROUP BY Product_ID, Product_Category;
```

Fact Table (Logical View)

```
CREATE OR ALTER VIEW vw_fact_sales
AS
SELECT
    ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS SalesKey,
    c.CustomerKey,
    p.ProductKey,

    s.Product_ID,
```

```
    s.Sale_Date,
    s.Region,

    s.Sales_Amount,
    s.Quantity_Sold,
    s.Discount,

    s.Payment_Method,
    s.Sales_Channel,
    s.SalesRep

FROM vw_sales_base s
JOIN vw_dim_customer c
    ON s.Customer_Type = c.Customer_Type
    AND s.Region = c.Region
JOIN vw_dim_product p
    ON s.Product_ID = p.Product_ID;
```

Step 5: Business-Level Views

These views provide aggregated insights for reporting.

Customer Sales Analysis

```
CREATE OR ALTER VIEW vw_customer_sales
AS
SELECT
```

```
    c.CustomerName,
    c.Region,
    SUM(f.Sales_Amount) AS TotalSales,
    SUM(f.Quantity_Sold) AS TotalQuantity
FROM vw_fact_sales f
JOIN vw_dim_customer c
    ON f.CustomerKey = c.CustomerKey
GROUP BY c.CustomerName, c.Region;
```

Product Performance

```
CREATE OR ALTER VIEW vw_product_performance
AS
SELECT
    p.Product_ID,
    p.Product_Category,
    SUM(f.Sales_Amount) AS Revenue,
    SUM(f.Quantity_Sold) AS UnitsSold,
    AVG(f.Discount) AS AvgDiscount
FROM vw_fact_sales f
JOIN vw_dim_product p
    ON f.ProductKey = p.ProductKey
GROUP BY p.Product_ID, p.Product_Category;
```

Sales Trend Analysis

```sql
CREATE OR ALTER VIEW vw_sales_trend

AS

SELECT

    CAST(Sale_Date AS DATE) AS SalesDate,

    SUM(Sales_Amount) AS DailyRevenue

FROM vw_sales_base

GROUP BY CAST(Sale_Date AS DATE);
```

Step 6: Security Configuration

- Created **Database Scoped Credential**
- Configured **External Data Source**
- Assigned appropriate **RBAC roles** (Storage Blob Data Reader)

Ensured secure access between Synapse and ADLS

```sql
CREATE DATABASE SCOPED CREDENTIAL ADLSCredential

WITH IDENTITY = 'Managed Identity';
```

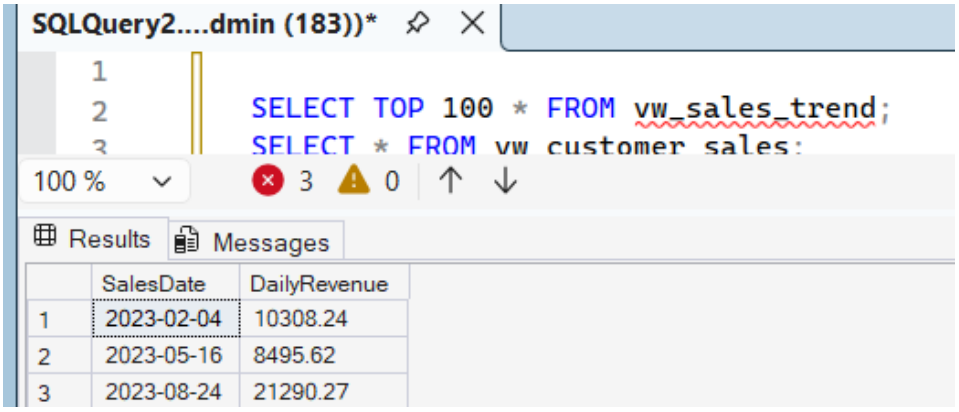
```sql
CREATE EXTERNAL DATA SOURCE sql_bc03_viewsPBI

WITH (

    LOCATION = 'https://adlsbk04.blob.core.windows.net/bootcamp03',

    CREDENTIAL = ADLSCredential

);
```

Step 7: SSMS Connectivity

- Connected using:

    - Synapse serverless endpoint and database credentials

- Queried views directly from SSMS
- Resolved credential-related access issues



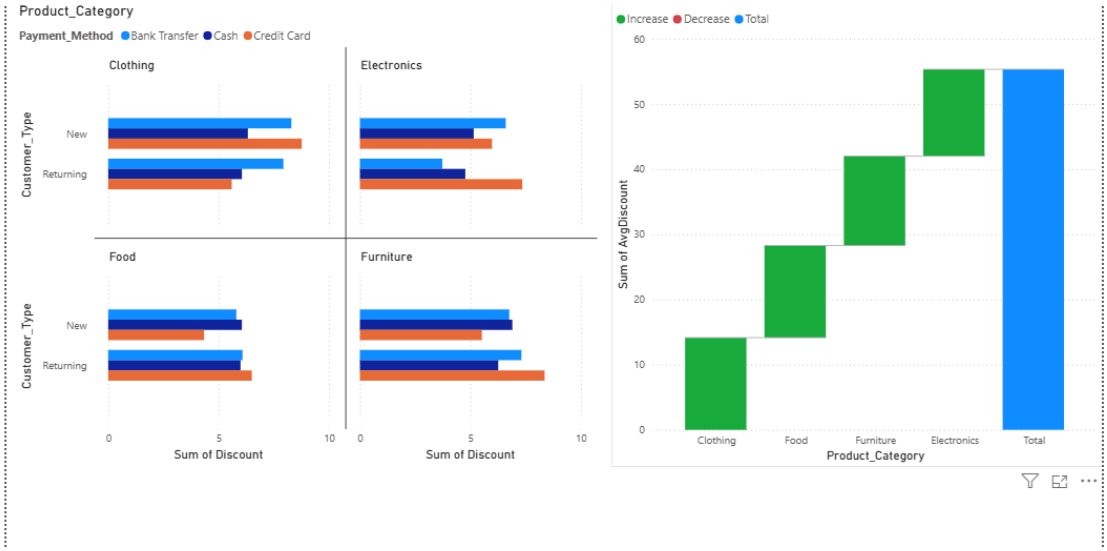
Step 8: Power BI Integration

- Connected using Azure Synapse connector
- Enabled DirectQuery mode (no data import)

Loaded Views:

- vw_customer_sales
- vw_product_performance
- vw_sales_trend

Enabled real-time reporting without data duplication



7. Dashboard Insights

The Power BI dashboard provides:

- Product performance analysis
- Daily sales trends

8. Challenges Faced

- Schema inconsistencies in Delta/Parquet files
- Data type mismatches (e.g., Long vs Double)
- Confusion between Serverless and Dedicated SQL pools
- Credential and permission errors in SSMS
- Limitations of external tables with Delta format

9. Solutions Implemented

- Enforced schema using Spark transformations
- Used OPENROWSET for flexible querying
- Implemented proper credential configuration
- Adopted serverless-only architecture

- Built logical star schema using views

10. Key Learnings

- Serverless SQL Pool enables true pay-per-query analytics
- Delta Lake ensures data reliability and performance
- Schema consistency is critical in distributed systems
- Views can effectively simulate dimensional modeling
- Security (RBAC + credentials) is essential in cloud environments

11. Final Conclusion

This project demonstrates a modern lakehouse architecture that combines:

- Serverless analytics (Azure Synapse)
- Scalable storage (ADLS Gen2)
- Optimized data format (Delta Lake)
- Real-time reporting (Power BI)

The solution delivers a highly scalable, cost-efficient, and production-ready analytics platform suitable for enterprise-level data workloads.